

PRODUCTION ARCHITECTURE • OMNICHANNEL RETAIL

Retail Price Recommendation Platform

Human-in-the-loop, not auto-pricing.
Built for item-store-week decisions with mandatory buyer approval.

Databricks Lakehouse | Demand Response | Constrained Optimization | Guardrails | Experimentation

- Item × Store × Week • Portal + Commercial Hub • Versioned & Auditable

For every Item × Store × Pricing Cycle

Platform does not start with "maximize price". It starts with decision support. Buyer retains final decision. System provides evidence.

Objective is configurable: Margin, Revenue, Inventory, Volume — not single metric

Approval mandatory: No recommendation publishes without buyer review

OUTPUT 1

1 Recommended price

Candidate from valid price ladder, not arbitrary extrapolation

OUTPUT 2

2 Expected demand

Demand(item,store,week | price) from response model

OUTPUT 3

3 Expected margin

(price − effective cost) × demand, with funding

OUTPUT 4

4 Expected sales

Revenue impact vs current: $Δ and %Δ

OUTPUT 5

5 Volume change

Predicted unit movement, inventory days cover

OUTPUT 6

6 Competitor position

Gap to matched competitor, availability flag

7 — CONFIDENCE

High / Medium / Low

Driven by data freshness, model certainty, inventory risk

8 — REASON

Why this price?

"Gap reduced + volume improves + margin above threshold"

Optimization objective — configurable by strategy

MAXIMIZE — WEIGHTED

Maximize: expected gross margin + revenue contribution + inventory objective + strategic volume objective

Not "maximize price". Trade-off explicit, tunable per business strategy, versioned.

WHY SCORING VS HARD RULES

Scoring allows trade-off, explainable, tunable per category. Hard rules block invalid, scoring ranks within allowed candidate set. Policy table stores weights w1-w6.

SUBJECT TO — GUARDRAILS

① price ≥ effective cost + minimum margin (effective-date-aware)

② movement ≤ approved category limit (±5% weekly typical)

③ must use approved price endings & ladder

④ KVI, legal, supplier agreement, store cluster rules

⑤ inventory availability + shrinkage/expiry

⑥ valid competitor match, no stale cost, buyer approval mandatory

Versioned: cost_version + competitor_snapshot + policy_version + guardrail_version

MARGIN STRATEGY

Higher w1 on gross margin. Use when cost pressure high.

VOLUME RECOVERY

Higher w on units + competitiveness. Close gap.

INVENTORY CLEARANCE

Higher w3 sell-through + expiry avoidance. Fresh.

MARKET PROTECTION

Higher penalty on competitor gaps. KVI trust.

END-TO-END • VERSIONED & AUDITABLE

Architecture — source to outcome

1. **Source Systems**
   POS, cost (effective-date), inventory, competitor, promo, supplier funding, buyer overrides
2. **Ingestion (Batch + Events)**
   Workflows / Auto Loader + Kafka cost stream. Freshness + full reconciliation.
3. **Databricks Lakehouse Bronze / Silver / Gold**
   Immutable history → conformed entities → decision-ready
4. **Feature → Demand → Optimization**
   Predict quantity per candidate, score with weighted objective + guardrails
5. **Recommendation → Portal → Buyer → Execution → Measurement**
   Portal review mandatory, no auto-publish. Outcome tracked.



VERSIONING AT EVERY STEP

model_version + feature_version + cost_version + competitor_snapshot + policy_version + guardrail_version

MLOPS FOUNDATION

MLflow + Unity Catalog (lineage) + Workflows + Delta.
Answers: Why 182?

Source systems — what feeds pricing

Every source needs **ingestion_timestamp + business_effective_timestamp + version**. For cost, upload time ≠ effective time. Keep both. Immutable history is truth.

POS SALES

item-store-date qty, revenue, discounts

Primary demand signal. Store cluster seasonality. 2+ years history for baseline.

COST

Unit cost effective-date aware

Most critical. Margin = f(cost active for recommendation period). Retain upload vs effective.

INVENTORY

On-hand, days cover, OOS rate

Shrinkage, expiry risk, lead time. Fresh needs more weight on waste avoidance.

PROMOTIONS

Calendar, display, coupon, funding

Regular vs promo must be separated. Funding impacts effective margin.

PRODUCT

Hierarchy, brand, private label, KVI

KVI flag drives competitor policy. Brand ladder influences price endings.

COMPETITOR

Matched item, gap, availability

Match quality validation required. Stale feed must block recommendation.

SHRINKAGE

Fresh waste, markdown risk

Expiry curve + waste %. Clearance strategy uses this heavily.

BUYER OVERRIDES

History of decisions & reasons

Feedback loop. Override rate is key decision quality metric. Never overwrite.

✓ source_system + source_record_id

✓ ingestion_timestamp vs effective_timestamp

✓ event_version + raw_payload kept in Bronze

Ingestion strategy — freshness + trust

Retail pricing fails on stale cost. Margin = (price − effective cost) × demand. Need both business effective timestamp and ingestion timestamp. One pipeline gives freshness, other gives trust.

BATCH — RECONCILIATION & HISTORY

Nightly full reconciliation

- POS sales, product hierarchy, promotion calendar, inventory snapshots
- Shrinkage, competitor refresh, price / cost history, Funding
- Fixes missed events, deduplication, late arrivals

Databricks Workflows / Lakeflow • Spark SQL + PySpark • Delta Lake • Auto Loader • Unity Catalog lineage

EVENT-DRIVEN — CRITICAL PATH

Cost change is critical event Kafka

Cost Tool → Kafka event → Streaming ingest → Validation → Recompute impacted item-store

- Why: margin depends on cost active for recommendation period
- Keep upload_ts vs effective_ts — they are different
- Only recompute affected pairs, don't rerun full portfolio

RECOMMENDATION

Stream gives freshness, batch repairs missed events. Event stream = incremental freshness < 15min for cost. Nightly batch = trusted reconciliation point. Both write Bronze with ingestion_timestamp + business_effective_timestamp + event_version.

Bronze — source history, little transformation

Immutable, append-only, full history. Keep both ingestion time and business effective time — critical for cost. Don't clean yet.

BRONZE TABLES — RAW PAYLOAD RETAINED

bronze.pos_sales	item-store-day qty, revenue
bronze.cost_history	RMS cost + effective_date
bronze.cost_events	Kafka stream, tool events
bronze.inventory_snapshot	on-hand, expiry risk
bronze.promotions	calendar, display, coupon
bronze.competitor_prices	matched competitor item
bronze.buyer_decisions	history of overrides

EVERY RECORD MUST INCLUDE

SOURCE_SYSTEM
POS / RMS / CostTool / Supplier feed

SOURCE_RECORD_ID
Original PK for lineage

INGESTION_TIMESTAMP
When we received it (Upload time)

BUSINESS_EFFECTIVE_TIMESTAMP
When cost is active (Effective time)

EVENT_VERSION
Cancellation / supersession handling

RAW_PAYLOAD
JSON / original row, no loss

⚠ **Critical for margin:** Upload timestamp ≠ Effective timestamp. For cost, retain both. If effective is future, margin calc for recommendation period must use correct future cost, not today's cost. Bronze makes this audit possible.

LAKEHOUSE • SILVER LAYER • CONFORMED, CLEANED, DEDUPLICATED

Silver — Conformed retail entities

SILVER TABLES

silver.item_store_sales_daily
POS daily grain, deduped by store+item+date

silver.item_store_cost_effective
Cost active for pricing cycle — critical

silver.item_store_inventory
On-hand, days cover, OOS rate, expiry risk

silver.item_store_competitor_price
Matched competitor item + availability

silver.item_store_promotion
Calendar, display, coupon, funding flag

silver.item_supplier_funding
Back-margin, supplier support per item

silver.price_history
Approved price + endings validated

silver.buyer_price_decision
Immutable buyer actions, versioned

COST EFFECTIVE LOGIC — NOT LATEST INGESTION

Business key: item_id + store_id + supplier_id + effective_date
Latest: ORDER BY source_updated_timestamp DESC
Take: approved + active + not cancelled + not superseded

Choose correct business-effective record, not newest ingestion. Consider cancellation, supersession, approval, activation status.

8
conformed entities feeding Gold — single source of truth for item-store

100%
effective-date-aware — margin depends on cost active for recommendation period

CDC
Deduplication via Delta MERGE + QUALIFY window logic

Audit
ingestion_ts + effective_ts + source_updated_ts + version retained

Common mistake: Picking latest ingestion_ts for cost. That breaks margin. You need effective_date for when cost is active, plus updated_ts for latest business version. Retain both.

QUALITY CHECKS IN SILVER

• No future effective_date without approval • No overlapping effective periods for same business key • Cost >0 and within supplier contract • Inventory snapshot = latest per store-day • Competitor match score > threshold

LAKEHOUSE • GOLD • DECISION-READY & AUDITABLE

Gold — decision-ready tables

GOLD TABLES (6)

- gold.pricing_features_item_store_week
- gold.demand_forecast_item_store_week
- gold.price_candidate_simulation
- gold.price_recommendation ← core
- gold.price_decision_audit
- gold.pricing_experiment_results

Full audit trail required for every row:
model_version + feature_snapshot_version + cost_version + competitor_snapshot_time + optimization_policy_version + guardrail_version → Answer: Why 182?

GOLD.PRICE_RECOMMENDATION — BUYER MUST SEE ALL BELOW

IDENTITY & CYCLE

- recommendation_id / pricing_cycle
- item_id + store_id + supplier
- current_price / final_price
- supplier_recommended_price

PREDICTION & IMPACT

- predicted_units_current → recommended
- predicted_revenue_change / Δ%
- predicted_margin_change / Δ%
- inventory_days_cover / expiry risk
- competitor_price + gap

CONFIDENCE & GOVERNANCE

- confidence_score H/M/L
- recommendation_reason (NL)
- guardrail_status / decision_status
- model_version / feature_version
- decision_timestamp / buyer_id

Immutable history: Never overwrite. Every system rec + supplier input + buyer override = own version. Buyer may freeze to earlier version — need version management.

Serving: Portal / Commercial Hub reads latest approved version per item-store-week. Downstream execution reads buyer_final_price only after approval flag = true.

FEATURE LAYER • DECISION-READY, AUDITABLE, VERSIONED

Feature layer — what model sees

Organized around price, demand memory, supply, commercial, context. All price-dependent features recalculated per candidate.

gold.pricing_features_item_store_week

$ PRICE

- current_price
- candidate_price
- log_candidate_price
- price_change_%
- price_vs_competitor
- price_index, discount_depth

□ DEMAND MEMORY

- lag_1, lag_2, lag_4
- rolling_4w, rolling_8w
- sales_momentum
- historical_price_percentile
- store cluster seasonality

□ SUPPLY & INVENTORY

- stock_on_hand
- days_of_supply
- out_of_stock_rate
- shrinkage, expiry_risk
- lead_time

% COMMERCIAL

- unit_cost (effective)
- supplier_funding
- promotion, display
- minimum_margin, brand
- private_label, KVI_flag

◂ CONTEXT

- store_cluster, region
- week_of_year, holiday
- payday, channel
- category
- competitor_availability

! **Critical:** For each candidate price, recalc price_vs_competitor, price_change_%, discount_depth, price_index. Model scores demand(item,store,week | candidate). Version feature_snapshot_version per recommendation.

CORE PRINCIPLE • PREDICT-THEN-OPTIMIZE

Don't predict best price directly — predict demand, then optimize

Demand model + price simulation + constrained optimization + business guardrails. Direct price prediction is unstable and not explainable. Response model is testable and tunable.

A

BASELINE DEMAND FORECAST — WITHOUT PRICE CHANGE

Qbaseline = f(history, seasonality, promo, inventory, store)

```
Qbaseline = f(
  historical sales, seasonality,
  holidays, promotions, inventory,
  store cluster, category, weather,
  recent demand, payday flag
)
```

Purpose: capture what would sell at current price. Isolate price effect from seasonality and promo lift. Baseline is needed to compute Δ vs candidate.

lag_1 / lag_2 / lag_4 | rolling 4w / 8w | seasonality | store cluster | holiday

B

PRICE-RESPONSE MODEL — CORE ELASTICITY

Demand(item, store, week | candidate price)

Estimate elasticity, not price point. For each candidate, recalculate price-dependent features, then predict.

log_candidate_price | price_vs_competitor | discount_depth | price_change_%

Start with **LightGBM or HistGradientBoosting** + log-linear baseline for interpretability. Hierarchical forecasting for sparse long-tail. Elasticity distribution monitored.

Why not direct? Direct "best price" model can't explain reason, can't swap objective weights, can't apply ladder constraints.

Recommended stack: LightGBM / HistGradientBoosting for tabular retail + log-linear baseline. CatBoost for high cardinality (store, category). Hierarchical for sparse SKUs. SHAP for reason codes.

LightGBM ★ start | HistGB | XGBoost | Log-Linear (interp)

CANDIDATE GENERATION • BOUNDED • LADDER-COMPLIANT

Generate only valid candidate prices

DON'T ALLOW UNCONSTRAINED EXTRAPOLATION

Exploration (model training):

```
candidates = [
  current*0.95,
  current*0.975,
  current,
  current*1.025,
  current*1.05
]
```

PRODUCTION — REAL PRICE POINTS (PRICE LADDER)

89.00 | **90.00 ← current** | 95.00 | 99.00 | 100.00

Rule: recalc price-dependent features for each p

price_vs_competitor, discount_depth, historical_percentile, price_index all recomputed per candidate before prediction.

FOR EACH CANDIDATE P IN ALLOWED SET

PREDICTED DEMAND

D(p) = model(p)

`Demand(item,store,week|p)`

REVENUE

R(p) = p × D(p)

`Candidate price × demand`

GROSS MARGIN

M(p) = (p - cost) × D(p)

`Effective cost at period`

IMPACTS

Vol, Rev, Margin, Inventory

`+ competitor gap`

Then select

recommended = argmax Score(p) for p in allowed_candidate_prices where guardrails pass. Score weights configurable by strategy (Margin / Volume / Clearance / Protection).

✓ **Bounded simulation** prevents unrealistic -30% drops. Ladder ensures shelf price is shoppable. All simulations stored in gold.price_candidate_simulation for audit.

OPTIMIZATION ENGINE • CONSTRAINED SCORING, NOT UNCONSTRAINED PREDICTION

Score(p) = weighted objective — argmax over valid candidates

CORE FORMULA

Score(p) =
w1 × expected_gross_margin(p)
+ w2 × expected_revenue(p)
+ w3 × inventory_clearance_value(p)
− w4 × expected_volume_loss(p)
− w5 × competitor_gap_penalty(p)
− w6 × price_change_penalty(p)

recommended_price = argmax Score(p)
for p ∈ allowed_candidate_prices

Why scoring vs hard rules: allows trade-off, explainable to buyer, tunable per strategy. Hard rules filter invalid; scoring ranks valid. All weights versioned in optimization_policy.

HOW IT WORKS IN PRODUCTION

For each item × store × week, generate only valid price ladder points (89, 90, 95, 99, 100). Recalculate price-dependent features, predict demand for each p, compute margin / revenue / gap / inventory impact. Pick p with highest Score. Show reason + confidence.

Margin
w1 high — protect gross margin when cost pressure

Volume Recovery
w2+w4 — favor units + competitiveness

Clearance
w3 high — sell-through, expiry avoidance fresh

Market Protect
w5 penalty high — KVI competitor trust

Confidence = data freshness (cost/competitor) × model certainty (elasticity stability) × inventory risk. Low confidence → no auto rec, buyer review only.

BUSINESS GUARDRAILS • MORE IMPORTANT THAN ALGORITHM

Guardrails — mandatory + confidence + category policy

MANDATORY — BLOCKS INVALID RECOMMENDATION

① price ≥ effective cost + minimum margin — no margin-negative publish

② movement ≤ approved category limit (e.g. ±5% weekly) — avoid shock

③ Must use approved endings & price ladder (89, 90, 95, 99, 100)

④ KVI policy, legal, supplier agreement, inventory availability check

⑤ No stale cost, no invalid competitor match, no publish without buyer approval

Guardrail failure = no recommendation. Logged in guardrail_status + reason_code for audit.

CONFIDENCE CONTROLS — DATA + MODEL CERTAINTY

HIGH — Fresh data, stable elasticity, trusted match → show preferred recommendation

MEDIUM — Partial staleness or higher variance → show with warning + reason

LOW — Stale cost / invalid match / sparse history → no auto rec, buyer review only

CATEGORY-SPECIFIC POLICY — ONE SIZE DOES NOT FIT ALL

Fresh / Perishable

Inventory, shrinkage, shelf-life matters more than margin % — prioritize waste avoidance

KVI — Known Value

Competitor position & customer trust matters more — tighter gap penalty

Long-tail

Sparse history — use pooled / category models, not individual store elasticities

Promotion

Separate regular vs promo decisions — funding changes effective margin

HUMAN-IN-THE-LOOP • PORTAL • BUYER RETAINS FINAL DECISION

Buyer reviews decision — UI must show evidence

CURRENT PRICE
190

SYSTEM RECOMMENDATION
182 ▼ 4.2%

SUPPLIER RECOMMENDATION
185

COMPETITOR / COST
180 / 160 gap -2, margin 22

PREDICTED UNITS
110 → 124 +12.7%

PREDICTED MARGIN
3,300 → 3,100 -6%, above thr

CONFIDENCE
Medium

GUARDRAIL
Pass • KVI ok • Ladder ok

Reason: competitor gap reduced, forecast volume improves +12 units, margin remains above threshold (cost 160 + min). Elasticity stable, cost fresh.

BUYER ACTIONS — MANDATORY APPROVAL

- ✓ Accept recommendation
- ↗ Override price
- — Hold current
- □ Reject
- □ Request recalculation

VERSIONING CRITICAL — DON'T OVERWRITE HISTORY

Every system rec + supplier input + buyer decision = immutable version. If override, capture:

override_price | override_reason | buyer_id | timestamp | source_rec_version

Problem: supplier rec arrives after buyer starts review → transaction frozen to earlier version. Need version mgmt & re-review trigger.

Rule: Must not publish without approval. Low confidence → no automated rec, buyer review only. All versions auditable via gold.price_decision_audit.

MLOPS • LINEAGE • AUDIT • WHY DID SYSTEM RECOMMEND 182?

MLOps — MLflow + Unity Catalog + Workflows

MLflow Experiments
Compare runs — demand MAE/WAPE, elasticity, bias per category
→ track

Model Registry / Unity Catalog
Approve production model versions — lineage + governance
→ approve

Databricks Workflows
Feature, training, scoring pipelines — batch + stream
→ run

Delta Tables (Gold)
Features, simulations, recommendations — immutable, versioned
→ serve

Unity Catalog + APIs
Permissions, audit, lineage — publish to portal
→ portal

EVERY RECOMMENDATION MUST RECORD

model_version : LightGBM v32
feature_version : snapshot 2025-05-11
cost_version : effective 2025-05-12 00:00
competitor_snapshot_time : 2025-05-11T22:00Z
optimization_policy_version : margin_strat v2
guardrail_version : KVI + ladder v4

This lets you answer: **"Why did system recommend 182 for this item at this store?"** — full lineage from raw Bronze → Silver effective → Gold features → model → Score(p) → guardrails → buyer decision.

PERMISSIONS

Unity Catalog: buyer sees rec + reason, cannot edit model. Engineer can deploy model, cannot approve price.

REPRODUCIBILITY

Feature snapshot version pinned at inference. Re-run possible years later with same inputs.

OBSERVABILITY • 4 LAYERS BEYOND FORECAST ACCURACY

Monitoring — data + model + decision + business

Don't judge success only by forecast accuracy. Judge by decision quality + business outcome. Each layer has owner and SLA.

DATA QUALITY — BLOCK BAD INPUT EARLY

- Missing cost / stale competitor price > 24h → block rec
- Duplicate item-store, invalid price, missing inventory
- Late promotion data, unmatched competitor item

MODEL QUALITY — DEMAND & ELASTICITY

- Demand MAE / WAPE, bias per category & store cluster
- Elasticity distribution drift, feature drift, prediction drift
- Low data flag → pooled / category model fallback

DECISION QUALITY — BUYER ADOPTION

- Acceptance rate, override rate, avg price movement
- Guardrail rejection rate, stability, time-to-decision
- Override reason clustering → policy tuning

BUSINESS OUTCOMES — REAL IMPACT

- Gross margin, revenue, units, basket value
- Market-price index, waste / shrinkage, stockouts, complaints
- Measured via controlled pilot only, not offline sim

! **Rule:** Low confidence or stale cost → no automated rec, buyer review only. Alert owner: data eng for data, DS for model, product for decision, merch for business.

EXPERIMENT DESIGN — PROVE CAUSALITY

Never claim uplift from offline simulation alone

Offline sim = observational. Production proof = controlled pilot. Label simulated uplift as observational, not causal.

 WHAT NOT TO DO

Offline simulation only

- Model predicts D(p) higher → claim margin +3% — **not causal**
- No buyer behavior, no inventory constraint, no competitor reaction in sim
- Leaks future promo info into training

✓ PRODUCTION VALIDATION

Controlled pilot — Treatment vs Control

T Treatment: buyer sees model recommendations in portal

C Control: existing pricing process

→ Randomize at **matched store-item groups** or **matched store clusters** to avoid contamination (same buyer, same category leakage)

EVALUATE — 6 METRICS + ADOPTION

BUSINESS (INCREMENTAL)

gross margin Δ | revenue Δ | units Δ | waste Δ | stockout Δ | competitor index

DECISION QUALITY

buyer adoption | acceptance rate | override rate | time to decision

PILOT GUARDRAILS

4–6 weeks minimum for retail

- Cover promotion cycle + payday + weekend seasonality
- Limit to 1–2 categories, limited stores to start
- Buyer approval mandatory — no auto execution yet

RULE **Simulated uplift = observational label in UI.** Only pilot / A/B test can be shown as incremental margin / revenue. This prevents false claims and builds buyer trust.

ROLLOUT • CRAWL WALK RUN — NO BIG BANG

Rollout — best practical for retail

Start manual and auditable, earn automation. Every phase keeps buyer approval and versioning.

1 CRAWL

DECISION SUPPORT

- 1-2 categories, limited stores, weekly batch
- Buyer approval mandatory, no auto exec
- Goal: trust + feedback

2 WALK

BETTER OPERATIONAL INPUTS

- Effective-date cost, competitor feed
- Inventory, shrinkage, supplier funding
- Versioned recommendations + audit

3 SCALE

EVENT-DRIVEN REFRESH

- Kafka cost events → recompute
- Impacted item-store pairs only
- Don't rerun whole portfolio

4 RUN

CONTROLLED AUTOMATION

- ✓ High confidence + stable elasticity
- ✓ Fresh data, no rule violations
- ✓ Repeated buyer acceptance + pilot proof

STRONGEST RECOMMENDATION

Databricks-based demand prediction + constrained optimization, fed by POS, effective cost, inventory, promo, competitor, published to portal for buyer approval and complete auditability. **Not one algorithm — trusted data + response model + optimizer + versioning + workflow + experimentation + monitoring.**

PRODUCTION READINESS • RETAIL PRICING PLATFORM

Technical centerpiece is not one algorithm

Trusted data + Demand response + Constrained optimizer + Version management + Buyer workflow + Experimentation + Monitoring & audit

Build as human-in-the-loop Price Recommendation Platform, not fully automatic AI changes shelf prices. Matches retail operating model: recommendations at item-store level, buyer sees suggestion with reason & confidence, can modify, retains final decision. All versions immutable & auditable.

SUCCESS = ADOPTION

Buyer acceptance + override learning

If buyers don't trust, no impact

SUCCESS = MARGIN

Incremental gross margin & revenue

Measured via controlled pilot only

SUCCESS = MEASURABLE CAUSALITY

Not offline sim uplift

4-6 weeks pilot, matched clusters

Databricks-based • Demand prediction + constrained optimization • Fed by POS, effective cost, inventory, promo, competitor • Portal approval • Complete auditability